

Practica 10 - Paradigmas de Lenguajes de Programacion

Objetivo: poder reconocer las diferentes características que poseen los paradigmas de programación

Los paradigmas representan diferentes formas de pensar y organizar el código para resolver problemas. Cada paradigma se adapta mejor a ciertos contextos.

Ejercicio 1: Un programa en un lenguaje procedural es una secuencia de instrucciones o comandos que se van ejecutando y producen cambios en las celdas de memoria, a través de las sentencias de asignación. ¿Qué es un programa escrito en un lenguaje funcional? y ¿Qué rol cumple la computadora?

En un **lenguaje funcional**, un programa no se basa en una secuencia de instrucciones que modifica el estado de la memoria (como en los lenguajes procedurales.), sino se basan en la **evaluación de funciones y expresiones matemáticas**.

Que es un programa escrito en un lenguaje funcional?

- Es una composición de funciones.
- No utilizan variables que cambian de valor.
- Se basa en el modelo matemático de funciones (como en cálculo lambda)
- Utiliza recursión en lugar de estructuras como bucles como for o while

Que rol cumple la computadora?

- Evalúa expresiones y aplica funciones.
- No sigue instrucciones paso a paso que alteran memoria como en un programa procedural.
- Actúa como un evaluador de funciones: Recibe una entrada, evalúa expresiones según reglas matemáticas y devuelve un resultado.

Ejercicio 2: ¿Cómo se define el lugar donde se definen las funciones en un lenguaje funcional?

En un lenguaje funcional, **las funciones se definen en el entorno o ámbito de definición** (en inglés, *environment*).

Esto puede ser:

- Un **módulo** o archivo.
- Un **bloque de código** (como en una expresión `let` o `where` en Haskell).
- El **entorno global** si la función está al nivel principal del programa.

Ejercicio 3: ¿Cuál es el concepto de variables en los lenguajes funcionales?

En lenguajes funcionales, una variable representa un valor fijo (es inmutable.)

A diferencia con los lenguajes imperativos, las variables en los lenguajes funcionales NO cambian de valor, mientras que en un lenguaje imperativo pueden cambiar de valor.

Ejercicio 4: ¿Qué es una expresión en un lenguaje funcional? ¿Su valor de qué depende?

Una expresión en un lenguaje funcional, es una combinación de funciones, valores, operadores y variables que pueden evaluarse para producir un resultado.

Ejemplo:

```
(3 + 4) * 2
```

Es una expresión que se evalúa a 14.

El valor de una expresión depende de sus argumentos (entradas), no depende del estado del programa ni de cambios en la memoria.

Siempre devuelve el mismo resultado para los mismos valores (esto se llama transparencia referencial).

Ejercicio 5: ¿Cuál es la forma de evaluación que utilizan los lenguajes funcionales?

Los lenguajes funcionales suelen usar evaluación perezosa (lazy evaluation), aunque algunos usan evaluación estricta (strict evaluation).

- Evaluación perezosa: Una expresión solo se evalúa cuando se necesita su valor.
- Evaluación estricta: Las expresiones se evalúan tan pronto como sean definidas.

Conclusión: La evaluación perezosa permite trabajar con estructuras infinitas y mejorar la eficiencia evitando cálculos innecesarios.

Ejercicio 6: ¿Un lenguaje funcional es fuertemente tipado? ¿Qué tipos existen? ¿Por qué?

Si, la mayoría de los lenguajes son **fuertemente tipados**, pero también hay algunos que son **estaticamente tipados**.

- **Fuertemente tipados:** no permite mezclar tipos de manera incorrecta (ejemplo sumar un número con una cadena)
- **Estaticamente tipado:** Los tipos se verifican en tiempo de compilación.

Los lenguajes funcionales modernos (como Haskell) son fuertemente tipados porque:

- Se basan en funciones puras, y los tipos ayudan a garantizar que las funciones reciban los argumentos correctos y devuelvan resultados válidos.
- El chequeo de tipos ayuda a prevenir errores comunes, ya que muchas veces los errores de lógica o mal uso de funciones se detectan en **tiempo de compilación**.
- Los lenguajes funcionales buscan ser matemáticamente correctos y previsibles, y el uso fuerte de tipos refuerza esto.

Conclusión: Los tipos fuertes en la programación funcional permiten mayor seguridad, legibilidad y corrección matemática.

Ejercicio 7: ¿Cómo definiría un programa escrito en POO?

Un programa escrito en **Programación Orientada a Objetos (POO)** se basa en **objetos que representan entidades del mundo real**, los cuales tienen **estado (atributos)** y **comportamiento (métodos)**.

El programa se estructura como una **colección de clases** que definen el molde para crear esos objetos.

Ejercicio 8: Diga cuáles son los elementos más importantes y hable sobre ellos en la programación orientada a objetos.

- **Clases:** Son plantillas o modelos que definen atributos y métodos.
- **Objetos:** Instancias de clases que encapsulan datos y comportamientos.
- **Encapsulamiento:** Oculta los detalles internos del objeto, exponiendo solo lo necesario.
- **Herencia:** Permite que una clase herede características de otra.
- **Polimorfismo:** Objetos diferentes pueden responder al mismo mensaje de manera distinta.
- **Abstracción:** Permite representar entidades complejas mediante modelos simplificados.

Ejercicio 9: La posibilidad de ocultamiento y encapsulamiento para los objetos es el primer nivel de abstracción de la POO, ¿cuál es el segundo?

El primer nivel es el encapsulamiento (ocultar datos internos)

El segundo nivel es la herencia, que permite modelar jerarquías y reutilizar código, definiendo

clases generales y derivadas mas especificas.

Ejercicio 10: ¿Qué tipos de herencias hay? Cuál usa Smalltalk y C++

Tipos de herencia:

1. **Simple**: una clase hereda de una sola clase.
2. **Múltiple**: una clase hereda de varias clases.
3. **Multinivel**: una clase hereda de otra, que a su vez hereda de otra.
4. **Jerárquica**: varias clases heredan de una misma clase base.
5. **Híbrida**: combinación de las anteriores.

Lenguajes:

- **Smalltalk**: usa **herencia simple**.
- **C++**: permite **herencia múltiple**.

Ejercicio 11: En el paradigma lógico ¿Qué representa una variable? ¿y las constantes?

- **Variables**: Representan valores desconocidos que serán unificados con valores concretos durante la resolución.
- **Constantes**: Representan **valores fijos** o hechos del dominio del problema (nombres, números, símbolos)

```
padre(juan, maria).
```

En este ejemplo, juan y maria son constantes.

Ejercicio 12: ¿Cómo se escribe un programa en un lenguaje lógico?

Se describe mediante:

- Hechos (base de conocimientos):

```
humano(socrates).
```

- Reglas (implicaciones):

```
mortal(X) :- humano(X).
```

- Consultas (preguntas al sistema):

```
?- mortal(socrates).
```

El lenguaje logico intenta satisfacer las consultas usando hechos y reglas mediante unificación y resolución.

Ejercicio 13: Teniendo en cuenta el siguiente problema, se lee una variable entera por teclado y si es par se imprime “El valor ingresado es PAR” y si es impar imprime “El valor ingresado es impar”, implemente este ejemplo en cada uno de los paradigmas presentados en esta práctica.

◆ Imperativo (Pascal):

```
program ejercicio13;
var
  numero : integer;
begin
  readln(numero);
  if (numero MOD 2 = 0) then
    writeln('El numero', numero, ' es PAR')
  else
    writeln('El numero', numero, ' es IMPAR');
end.
```

◆ Funcional (Haskell):

```
main = do
  input <- getLine
  let n = read input :: Int
  putStrLn (if even n then "El valor ingresado es PAR" else "El valor
ingresado es impar")
```

◆ Orientado a objetos (Java):

```
public class Ejercicio13 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        if (n % 2 == 0)
            System.out.println("El valor ingresado es PAR");
        else
```

```

        System.out.println("El valor ingresado es impar");
    }
}

```

◆ Lógico (Prolog):

```

par(N) :- 0 is N mod 2.
impar(N) :- 1 is N mod 2.

mensaje(N) :- par(N), write('El valor ingresado es PAR').
mensaje(N) :- impar(N), write('El valor ingresado es impar').

% Consulta:
% ?- mensaje(4).

```

Ejercicio 14: Describa las características más importantes de los Lenguajes Basados en Scripts. Mencione diferentes lenguajes que utilizan este concepto. ¿En general, qué tipificación utilizan?

- Interpretados:
Los scripts no necesitan compilarse. Son ejecutados por un interprete, lo que permite una ejecucion inmediata y facilita el desarrollo rapido.
- Facilidad de uso:
Estan diseñados para ser simples de escribir y entender, con una sintaxis amigable y menor cantidad de codigo necesario para realizar tareas comunes.
- Automatizacion:
Se usan frecuentemente para automatizar tareas repetitivas, especialmente en entornos como administracion de sistemas, desarrollo web o pruebas automaticas.
- Tipado dinamico:
No se necesita declarar el tipo de las variables. El tipo se determina en tiempo de ejecucion.

Lenguajes comunes que utilizan scripts:

- **Python** 🐍
- **JavaScript** (tanto en frontend como en backend con Node.js)
- **Bash** (Shell scripting en Linux/Unix)
- **Ruby**
- **PHP**
- **Perl**
- **Lua**

- **PowerShell** (en entornos Windows)

En general, los lenguajes basados en scripts utilizan tipado dinámico.

- El tipado dinámico se determina automáticamente en tiempo de ejecución.

```
x = 5
x = "Hola"
```

Ejercicio 15: ¿Existen otros paradigmas? Justifique la respuesta

Si, existen otros paradigmas además de los que vimos en esta práctica (imperativo, funcional, orientado a objetos y lógico).

Los paradigmas representan diferentes formas de pensar y organizar el código para resolver problemas. Cada paradigma se adapta mejor a ciertos contextos.

◆ Algunos otros paradigmas importantes son:

1. Paradigma orientado a eventos

- Se basa en la **respuesta a eventos**, como clics del mouse, teclas presionadas o mensajes de red.
- Muy usado en **interfaces gráficas**, programación web y aplicaciones interactivas.
- Lenguajes que lo usan: **JavaScript**, **C# (WinForms, WPF)**, **Java (Swing, JavaFX)**.

2. Paradigma reactivo

- Centrado en el **flujo de datos y la propagación de cambios**.
- Es ideal para manejar **datos asíncronos** o cambios en tiempo real.
- Ejemplo: si una variable cambia, otras que dependen de ella se actualizan automáticamente.
- Tecnologías: **RxJS**, **ReactiveX**, **Recoil**, **Jetpack Compose**.

3. Paradigma declarativo

- El **qué hacer** es más importante que el **cómo hacerlo**.
- No describe pasos exactos, sino relaciones, reglas o condiciones.
- Subconjuntos: **funcional**, **lógico**, **SQL**, **HTML**.

4. Paradigma orientado a aspectos (AOP)

- Permite separar **comportamientos transversales** (como logging, seguridad, auditoría) del código principal.
- Mejora la modularidad.
- Lenguajes/tecnologías: **AspectJ** (para Java), **Spring AOP**.

5. Paradigma basado en componentes

- En lugar de centrarse en funciones u objetos, se construyen **componentes reutilizables**.
- Muy usado en desarrollo moderno de interfaces (por ejemplo, **React**, **Blazor**, **Angular**).

◆ ¿Por qué existen tantos paradigmas?

Porque cada paradigma:

- Aborda los problemas desde una **perspectiva diferente**.
- Facilita la resolución de ciertos tipos de problemas (ej. concurrencia, reactividad, interfaces, lógica formal).
- Evoluciona con las necesidades del desarrollo de software (rendimiento, mantenibilidad, escalabilidad).